

Deep Learning Final Project:
Fetal Head Detection in Ultrasound Images
Adrià García, Manel Gutierrez & Ernest Fosch

INDEX

1. Introduction and motivation:.....	1
2. Related work:.....	1
3. Dataset and preprocessing:.....	2
4. Methodology and implemented models:.....	3
5. Experiments and evaluated configurations:.....	4
6. Results and Discussions.....	6
7. Comparison against Wang's approach.....	8
8. Discussion of limitations and observations.....	9
9. Conclusions and future work.....	9
10. References:.....	10

1. Introduction and motivation:

Monitoring fetal growth during pregnancy is one of the most important aspects of prenatal care. Among the measurements taken during routine ultrasound scans, the fetal head circumference stands out as a key indicator of healthy brain development and overall fetal growth.

Currently, this measurement is performed manually by a sonographer, who must locate the correct perimeter of the fetal skull and carefully trace the ellipse manually. In this process, the accuracy depends heavily on the experience and concentration of the person performing it. Two sonographers examining the same scan can arrive at noticeably different measurements, which can lead to inconsistent clinical decisions.

By automating this process with artificial intelligence we believe that the human error can be minimized, and the measurement can achieve a more accurate and consistent result while at the same time making the task less time-consuming for the ultrasound technician.

2. Related work:

We will base this project on the fetal head ultrasound repository of Fangyijie Wang. In his implementation, he used a U-net architecture whose encoder is replaced by a MobileNet V2 pre-trained on ImageNet to create a model which is able to segmentate the fetus head. For each pixel, the network outputs the probability of the pixel to be part of the head and this probability map is then converted into a binary mask by applying a threshold. Finally, the mask is then used to generate an ellipse by applying a Hough Transform.

The U-net architecture:

U-Net is a convolutional neural network architecture which uses a symmetric encoder-decoder with skip connections structure which is designed for image segmentation problems as the ultrasound segmentation.

The basic block used throughout the network is the DoubleConv block which consists in two 2D convolutions in cascade each one followed by a batch normalization and a ReLU activation function as so:

$$Input \rightarrow Conv2d \rightarrow BatchNorm \rightarrow ReLU \rightarrow Conv2d \rightarrow BatchNorm \rightarrow ReLU \rightarrow Output$$

The idea is to first, in the encoder part, perform a series of maxpooling operations to reduce the size of the image by two progressively. Therefore, a 256×256 image becomes 128×128, then 64×64, and so on down to 16×16 at the bottleneck. At the same time, the number of channels of the image doubles at each pooling operation from 64ch to 1024ch. As we lose definition, the network needs more feature maps to compensate.

The bottleneck is the deepest and more compressed point of the network. At this point we have a 16x16 with 512 channels, and we perform the same double convolution block but

without doing pooling so this results in a 16x16 image with 1024 channels. The decoder then takes this representation and starts the reverse process, progressively upsampling back to the original resolution while using the skip connections to recover the spatial detail that was lost along the way to generate the final probability for each pixel to be part of the segmented object.

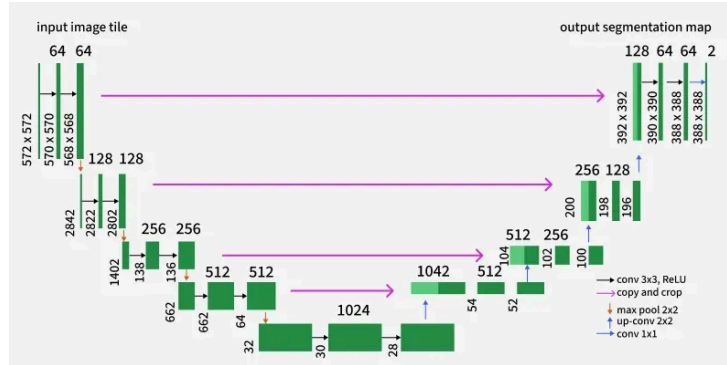


Figure 1: U-Net architecture diagram.

3. Dataset and preprocessing:

We used the same dataset as Wang's. It was taken from the HC18 challenge and consists in a training set of 799 images and a test set of 200 images. The training set was randomly split into 80% for training and 20% for validation. However, the same seed was always used for reproducibility.

Each sample consists of an ultrasound image and a segmentation mask where the pixels that belong to the fetal head are white, and the rest is black. The ground truth mask is always represented as the ellipse that best fits the head, but since our goal is that our network performs a regression task rather than a segmentation task, that is, to predict this ellipse instead of classifying each pixel as fetal head or not, we need a way to represent an ellipse with a few values. We noted that in Wang's work, once the mask was predicted it used some OpenCV functions to find the contours of the mask (`cv2.findContours`) and to fit an ellipse given those contours (`cv2.fitEllipse`). This last function returns 5 parameters that define an ellipse: the center in pixel coordinates (c_x , c_y), the major and minor diameters (d_{max} , d_{min}) also expressed in pixels, and the rotation angle (θ). Therefore, we used these functions to compute the ground truth ellipse parameters based on the provided masks.

The ellipse parameters (c_x , c_y , d_{max} , d_{min}) are expressed in pixel units and their numerical ranges depend on the image dimensions. To make the regression problem better conditioned, these values were normalized by the mask size, resulting in target values approximately bounded between 0 and 1. This prevents some parameters from having much larger magnitudes than others and ensures that all output dimensions contribute more evenly to the loss function. Such normalization is known to improve numerical stability during training and often accelerates convergence of gradient-based optimization methods.

Additionally, the ellipse orientation was represented by its sine and cosine components instead of a single angular value. Direct angle regression suffers from a discontinuity at the

wrap-around boundary (e.g., 0° and 360° represent the same orientation but are numerically far apart), which can lead to misleadingly large regression errors. By encoding the orientation as $(\sin\theta, \cos\theta)$, geometrically similar orientations remain close in the target space, yielding a continuous representation that improves optimization stability and facilitates network training. Since an ellipse has a 180° rotational symmetry, `cv2.findEllipse` returns an angle between $[0, 180)$. Therefore, instead we encoded $(\sin 2\theta, \cos 2\theta)$.

As we can see in figure 2, after denormalizing the obtained ellipse parameters from the ground truth mask, we can draw an ellipse on top of the original ultrasound image to verify that the target values are being computed correctly.

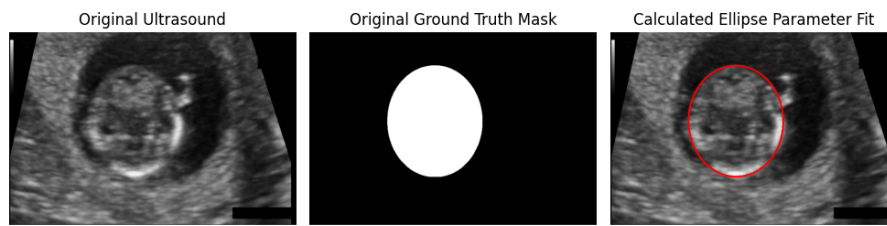


Figure 2: Original ultrasound image and mask, drawn ellipse from recovered parameters.

4. Methodology and implemented models:

As explained in section 3, for our implementation of this head detection task, we chose to change the mask approach of Wang's research for a linear regression of the 6 key parameters that define an ellipse.

We started by choosing the MobileNet V3 small model pre-trained on ImageNet as our starting point and the only part of the model we had to modify was the final classifier layer. MobileNet originally has a classifier designed to predict up to 1000 categories so we had to substitute it for a linear classifier that outputs the 6 values we want. Before this we also added a fully connected layer that compresses the 576 features coming out of the MobileNet down to 128 values, a Hardswish activation function to limit the model learning and a dropout of 0.2 which randomly disables 20% of the neurons to prevent overfitting. Finally, a last linear layer takes those 128 values and predicts our 6 target parameters.

MobileNet v3 architecture:

The version 3 of MobileNet provides two models, one targeted for high resource usage and one for low: MobileNetV3-Large and MobileNetV3-Small. Both of these networks are built with several MoblinetV3 blocks as the one shown in figure 3.

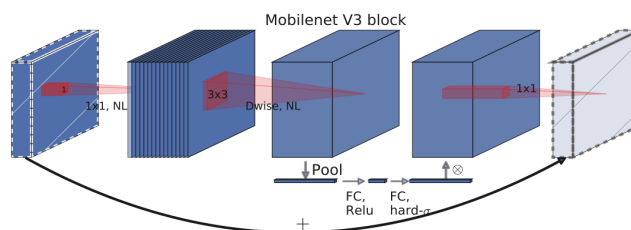


Figure 3: MobileNet v3 block diagram.

Each block is built with: first a 1x1 convolution which expands the number of channels, then a 3x3 depthwise convolution which applies the filter channel by channel, instead of the standard fully connected convolution, reducing the number of parameters needed.

After this, we have a Squeeze-and-Excitation module which does a pooling followed by two fully connected convolutions layers one with ReLU and the other with hard-sigmoid activation function to later be rescaled to the original feature map to then end the block with another pointwise 1x1 convolution which now compresses and reduces the number of features and uses no activation function.

As said earlier, both small and large architectures stack this block to build its structure. Specifically in figure 4 we can see the layers each one has and how many blocks are using:

Input	Operator	exp-size	ϕ_{out}	SE	NL	s
$224^2 \times 3$	conv2d	-	16	-	HS	2
$112^2 \times 16$	bneck, 3x3	16	16	-	RE	1
$112^2 \times 16$	bneck, 3x3	64	24	-	RE	2
$56^2 \times 24$	bneck, 3x3	72	24	-	RE	1
$56^2 \times 24$	bneck, 5x5	72	40	✓	RE	2
$28^2 \times 40$	bneck, 5x5	120	40	✓	RE	1
$28^2 \times 40$	bneck, 5x5	120	40	✓	RE	1
$28^2 \times 40$	bneck, 3x3	240	80	-	HS	2
$14^2 \times 80$	bneck, 3x3	200	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	184	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	184	80	-	HS	1
$14^2 \times 80$	bneck, 3x3	480	112	✓	HS	1
$14^2 \times 112$	bneck, 3x3	672	112	✓	HS	1
$14^2 \times 112$	bneck, 5x5	672	160	✓	HS	2
$7^2 \times 160$	bneck, 5x5	960	160	✓	HS	1
$7^2 \times 160$	bneck, 5x5	960	160	✓	HS	1
$7^2 \times 160$	conv2d, 1x1	-	960	-	HS	1
$7^2 \times 960$	pool, 7x7	-	-	-	-	1
$1^2 \times 960$	conv2d 1x1, NBIN	-	1280	-	HS	1
$1^2 \times 1280$	conv2d 1x1, NBIN	-	k	-	-	1

Table 1: MobliNetV3-Large

Input	Operator	exp-size	ϕ_{out}	SE	NL	s
$224^2 \times 3$	conv2d, 3x3	-	16	-	HS	2
$112^2 \times 16$	bneck, 3x3	16	16	✓	RE	2
$56^2 \times 16$	bneck, 3x3	72	24	-	RE	2
$28^2 \times 24$	bneck, 3x3	88	24	-	RE	1
$28^2 \times 24$	bneck, 5x5	96	40	✓	HS	2
$14^2 \times 40$	bneck, 5x5	240	40	✓	HS	1
$14^2 \times 40$	bneck, 5x5	240	40	✓	HS	1
$14^2 \times 40$	bneck, 5x5	120	48	✓	HS	1
$14^2 \times 48$	bneck, 5x5	144	48	✓	HS	1
$14^2 \times 48$	bneck, 5x5	288	96	✓	HS	2
$7^2 \times 96$	bneck, 5x5	576	96	✓	HS	1
$7^2 \times 96$	bneck, 5x5	576	96	✓	HS	1
$7^2 \times 96$	conv2d, 1x1	-	576	✓	HS	1
$7^2 \times 576$	pool, 7x7	-	-	-	-	1
$1^2 \times 576$	conv2d 1x1, NBIN	-	1024	-	HS	1
$1^2 \times 1024$	conv2d 1x1, NBIN	-	k	-	-	1

Table 2: MobliNetV3-Small

Figure 4: Both MobliNetV3 models architectures.

5. Experiments and evaluated configurations:

MobileNet transfer learning:

For the first training we decided to freeze the entire MobileNetV3 small backbone and only the custom linear classifier that we added is left trainable. As a loss function we used a smooth L1 loss which behaves as a mean squared error for small errors but becomes linear for larger ones. This function is useful because it reduces the gradients on small errors and therefore the model is able to focus more on trying to optimize the weights to reduce larger errors. Additionally, we chose an Adam optimizer with a learning rate of 0.001 and performed the training for 15 epochs without data augmentation.

This first experiment serves as a baseline to understand how well the frozen ImageNet weights alone can generalize to our head ellipse regression task, before exploring more aggressive fine-tuning strategies.

Fine-Tuning:

For the second training we decided to implement two fine-tuning strategies by selectively unfreezing deep layers of the MobileNetV3 feature extractor. We make this approach to train the last convolutional layers too, for this approach we explored two different strategies:

- **Moderate Fine-Tuning:** In this strategy we unfreeze only the last two layers of the feature extractor, the final Inverted Residual block and the final convolutional layer. This last convolutional layer expands the feature maps from 96 to 576 channels right before the global average pooling step. By unfreezing only these final two layers, we allow the model to adjust how it aggregates and projects the highest-level abstract representations into the final 576-dimensional embedding that our regression classifier reads. The early and middle blocks remain completely frozen, retaining their robust, pre-trained ImageNet feature extractors.
- **Aggressive Fine-Tuning:** In this approach we unfreeze the last six layers. This range includes several consecutive Inverted Residual blocks that gradually transition the network from mid-level feature processing to high-level semantic mapping. The ultrasound images present unique challenges, such as noise, which differ drastically from natural ImageNet photos. With this strategy the network has the opportunity to reconstruct and adapt mid-to-high-level geometric shapes, such as the continuous curvature of the fetal cranium, while still shielding the earliest convolutional layers.

When we are training with these strategies, as a loss function we use a smooth L1 loss as previously mentioned, but when choosing the optimizer we also use the Adam, but assigning a higher learning rate (1e3) for the classifier and a lower one (1e5) for the last conv. blocks.

Data Augmentation:

After applying the Aggressive Fine-Tuning strategy, we decided to implement Data Augmentation to mitigate overfitting and improve the generalization of our model. First we resize and normalize all images to match the expected input distribution of the pre-trained MobileNetV3 backbone. To improve robustness to the variability inherent in ultrasound acquisition, we introduce geometric augmentations that simulate realistic variations in probe positioning. We apply a random rotation up to $\pm 15^\circ$, which reflects the natural variability in the orientation of the transducer during scanning. This is particularly relevant because the fetal head can appear rotated in many different directions depending on the sonographer. The same transformation is applied consistently to both the ultrasound image and the corresponding mask to preserve the geometric relationship between the image content and the target ellipse parameters. In addition, we apply a random affine transformation with translations up to 10% of the image size and scaling between 0.9 and 1.1, which simulates small shifts in probe placement and variations in zoom or imaging depth. These augmentations are essential for learning translation- and scale-invariant representations, which directly improve the stability of the predicted center coordinates and ellipse axes. We also apply brightness and contrast jittering, which exposes the model to both brighter and darker images and finally Gaussian blur to introduce some noise and make the learning task more difficult.

MobileNetv3_Large:

For the last training, we decided to replicate our initial baseline approach but utilizing the MobileNetV3-Large backbone instead. Just as before, we froze the entire pre-trained feature extractor, leaving only our custom regression classifier trainable. As a loss function, we used

the same smooth L1 loss, and we kept the Adam optimizer with a learning rate of 0.001, performing the training for 15 epochs without data augmentation.

We used the MobileNetV3-Large architecture as is designed for higher resource usage. It features a deeper network with more Inverted Residual blocks and wider channel dimensions than the Small version. This makes the model more complex which helps to extract richer, more detailed features from the ultrasound images, and it is translated into a more accurate prediction of the fetal head ellipse compared to the lightweight version.

6. Results and Discussions



Figure 5: Training and Validation loss curves for every experiment explained in section 5. Ex. 1 to Ex. 5 in order.

	Ex. 1	Ex. 2	Ex. 3	Ex. 4	Ex. 5
mSmoothL1Loss	0.0202	0.0241	0.0245	0.0193	0.0155
MAE	0.14	0.14	0.14	0.13	0.11
mIoU	0.4533	0.5118	0.4861	0.5696	0.6309
Trainable Parameters (M)	0.075	0.425	0.811	0.863	2.904
Total Parameters (M)	1.002	1.002	1.002	1.002	3.096

Table 1: Mean SmoothL1Loss, MAE and Mean IoU on the test set for every model obtained in each experiment explained in section 5.

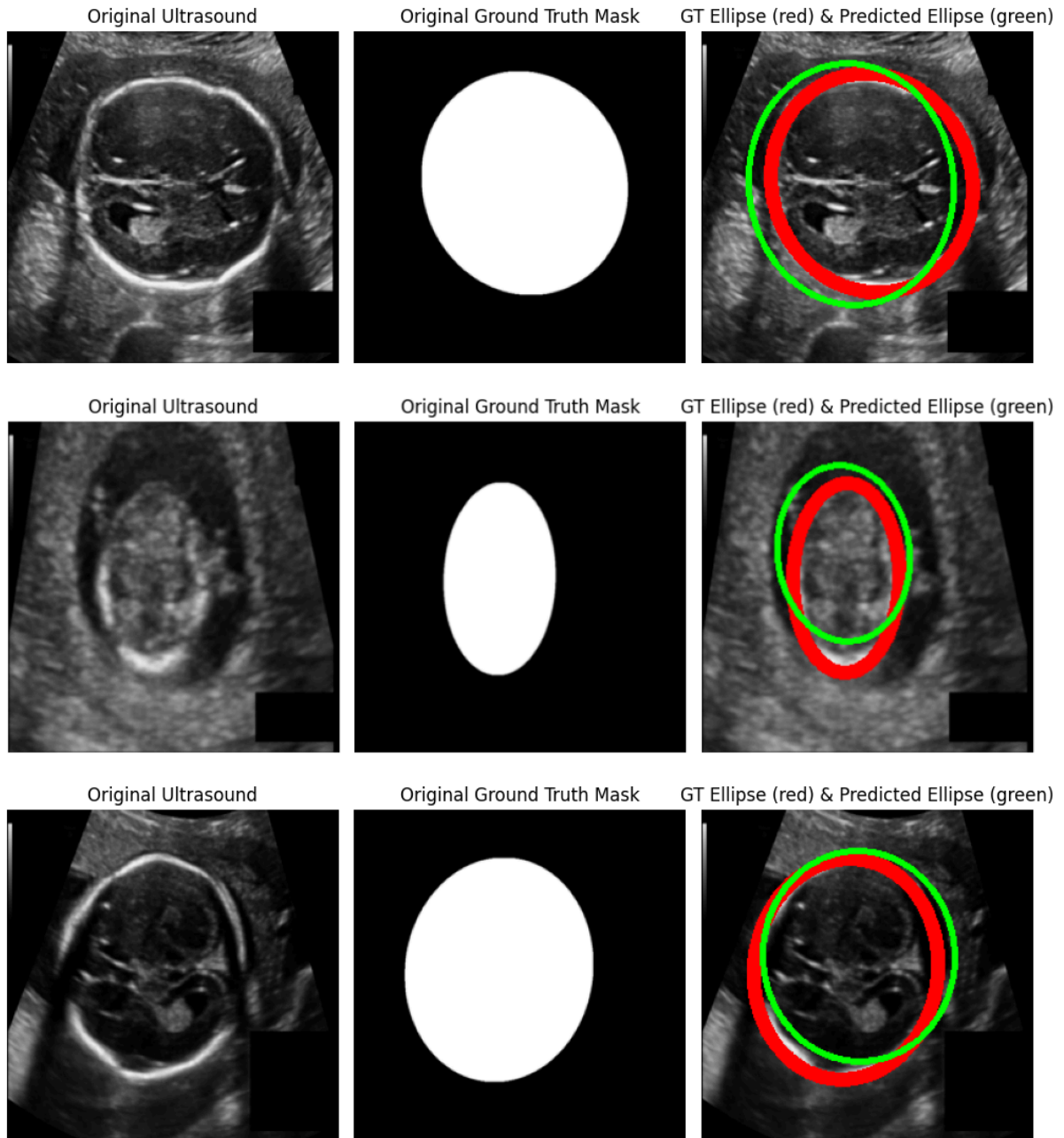


Figure 6: Some of the predictions of our “best” model achieved in Ex. 6.

Ex. 1: The Base Model (Training just the Classifier/Regression Head)

By keeping the MobileNetV3 backbone fully frozen and training only the regression head, the model relies entirely on ImageNet-pretrained features without any adaptation to ultrasound imaging characteristics. While these features are powerful for natural images, they are not optimally aligned with the low-contrast structure of fetal head ultrasound scans.

As a result, the regression head must learn a complex mapping from features that are not fully discriminative for ellipse geometry, which leads to overfitting as the model starts updating the trainable weights so that they perform well on the training set. Consequently,

performance remains poor, with the model achieving a mIoU of 0.4533 and relatively stable but suboptimal error values in the training loss curve.

Ex. 2 & Ex. 3: Moderate vs. Aggressive Fine-tuning:

In Ex. 2 (moderate fine-tuning), unfreezing the last two convolutional layers of the backbone allows the network to adjust the highest-level feature representations while preserving the general low- and mid-level filters learned from ImageNet. This results in a significant improvement in segmentation quality, with mIoU increasing to 0.5118, indicating that the model is better aligning with the ellipse regression task.

However, when even more layers are unfrozen (Ex. 3), the IoU decreases from 0.5118 to 0.4861 despite the increase in trainable parameters. This suggests that the additional model capacity does not necessarily translate into better generalization. With a limited training dataset, unfreezing a larger portion of the backbone may lead to overfitting, causing the network to adapt excessively to the training samples and lose some of the robust feature representations learned during pretraining.

Ex. 4 & Ex. 5: Data Augmentation and Large Backbone Effect

With data augmentation in Ex. 4, the model becomes more robust, achieving a higher mIoU of 0.5696. However, training remains somewhat unstable, as reflected in fluctuating loss curves. This suggests that although augmentation reduces overfitting, the smaller MobileNetV3 backbone still has limited capacity to fully model the variability of the data.

Finally, in Ex. 5, switching to MobileNetV3-Large leads to the best overall performance, with mIoU reaching 0.6309 and MAE improving to 0.11. The larger backbone provides a much stronger feature extractor, allowing better extraction of geometric structures such as the fetal head boundary while maintaining better stability during optimization.

7. Comparison against Wang's approach

Backbone	Trainable Params (MM)	Trainable Layers	Model Size (MB)	Epochs	Learning Rate	Pixel Accuracy	Dice Score	mIoU
None	31	E and D	118.5	20	0.0001	97.33 (0.39)	95.56 (0.62)	91.56 (1.16)
MobileNet_v2*	4.4	D - 0,1,2,3,4	59.2	20	0.0001	91.68 (1.09)	86.68 (1.06)	76.70 (1.68)
MobileNet_v2*	2.22	E - 0,1,2,3,4	42.7	20	0.0001	90.11 (2.60)	85.67 (3.10)	75.14 (4.69)
MobileNet_v2	2.22	E - 0,1,2,3,4	42.7	20	0.0001	97.65 (0.44)	96.22 (0.68)	92.73 (1.26)
MobileNet_v2	3.76	D - 0	54.3	20	0.0001	87.00 (8.55)	82.46 (8.89)	70.99 (12.63)
MobileNet_v2	4.24	D - 0,1	57.9	20	0.0001	95.81 (1.74)	93.27 (2.46)	87.53 (4.20)
MobileNet_v2	4.36	D - 0,1,2	58.9	20	0.0001	97.16 (0.11)	95.34 (0.14)	91.13 (0.26)
MobileNet_v2	0.164	D - 2,3,4	26.8	20	0.0001	88.88 (0.50)	81.46 (0.53)	68.85 (0.77)
MobileNet_v2	0.00698	D - 4	25.6	20	0.0001	68.31 (1.01)	37.04 (6.31)	22.90 (4.61)
MobileNet_v2	4.4	E - 0,1,2,3,4	59.2	20	0.0001	97.77 (0.22)	96.28 (0.38)	92.87 (0.69)

Figure 7: Wang's results on different number of trainable parameters and layers.

We can see in Figure 7 that Wang obtained his best results when training the first 4 layers of the encoder (MobileNet_v2), which corresponded to a total of 4.4 million trainable

parameters. It seems counterintuitive at first as usually, in transfer learning, we freeze the early layers and fine-tune the deep ones. We believe that Wang does the exact opposite here for a brilliant reason unique to U-Net medical segmentation: U-Net relies on skip connections to recover spatial information as downsampling operations compress the image, erasing fine details and spatial localization. Therefore, skip connections reintroduce this localized data so the network can construct highly precise segmentation masks.

8. Discussion of limitations and observations

First, we think that the dataset size might be relatively small compared to the capacity of the models, which contributes to unstable training dynamics and sensitivity to hyperparameters. Since we had very small batches the gradients had a very high variance leading to an unstable training because each image has a stronger influence on parameter updates. Also, with few samples and very outside from the image domain where the MobileNetv3 was trained it is possible that the pretrained weights learnt from ImageNet are being distorted instead of refined, which is why Ex. 3 slightly worsens performance. Second, the regression formulation of ellipse parameters is inherently sensitive, since small coordinate errors can lead to large variations in IoU. Finally, even with data augmentation, the variability of ultrasound imaging is not fully captured therefore generalization is not achieved.

9. Conclusions and future work

We did not obtain the results we wanted to achieve because we used the MobileNet, which is an efficient network and therefore underperforms on ultrasound regression tasks where accuracy is actually more important than efficiency. This happens because it is too lightweight and too classification oriented. Also, using a pre-trained MobileNet model on ImageNet was not the best idea because this dataset does not contain ultrasound images. Therefore, with our small training dataset the model was not able to learn enough the features of our input images.

Our future work would be to improve our results by using either a heavier CNN architecture such as ResNet, or a model pre-trained on medical imaging datasets, in order to better capture domain-specific features present in ultrasound images. This would allow us to compare the performance of general pre-trained networks, like the one we tested, against models trained on medical data, and evaluate whether domain-specific pre-training leads to more accurate and robust ellipse regression results.

10. References:

GitHub Repository: FT Methods for Fetal Head Segmentation,
2023. https://github.com/13204942/FT_Methods_for_Fetal_Head_Segmentation

PyTorch Contributors,
“torchvision.models.mobilenet_v3_small — PyTorch Documentation,”
https://docs.pytorch.org/vision/main/models/generated/torchvision.models.mobilenet_v3_small.html

PyTorch Contributors,
“torchvision.models.mobilenet_v3_large — PyTorch Documentation,”
https://docs.pytorch.org/vision/main/models/generated/torchvision.models.mobilenet_v3_large.html